

Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 5

- Recap on correlation screening for bike-share data
- Apply mutual information screening (another filter method)
- Mutual information
- Conditional correlation screening

Recall: Filter methods and correlation screening

- Defined as feature selection method that is independent of prediction function f
- Correlation screening is one of the simplest filter methods
- Computationally cheap since simply involves summary statistics and no matrix inversions
- Not too data intensive since we are only estimating summary statistics

Recall: Correlation screening for bike-share data

```
[20]: rej, p_adj, _, _ = multipletests(
    corr_df["p_value"],
    alpha=0.05,
    method="fdr_bh"
)

corr_df["p_adj"] = p_adj
corr_df["significant"] = rej
corr_df
```

```
[20]:
```

	feature	correlation	abs_correlation	p_value	p_adj	significant
1	atemp	0.631066	0.631066	1.854504e-82	1.854504e-81	True
0	temp	0.627494	0.627494	2.810622e-81	1.405311e-80	True
5	yr	0.566710	0.566710	2.483540e-63	8.278466e-63	True
4	season	0.406100	0.406100	2.133997e-30	5.334992e-30	True
9	weathersit	-0.297391	0.297391	2.150976e-16	4.301952e-16	True
6	mnth	0.279977	0.279977	1.243112e-14	2.071853e-14	True
3	windspeed	-0.234545	0.234545	1.359959e-10	1.942798e-10	True
2	hum	-0.100659	0.100659	6.454143e-03	8.067679e-03	True

Limitations of correlation screening as a feature selection method

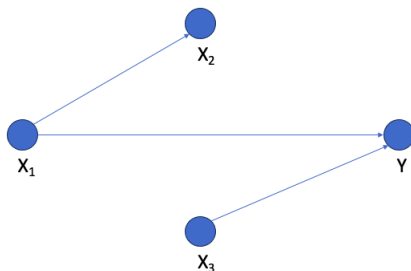
Limitations of correlation screening as a feature selection method

- Captures *marginal* dependence relationships, not conditional independence
- Only captures linear relationships

Capturing marginal vs conditional dependence

Recall housing data/linear model example from earlier.

$$\begin{aligned} Y &= X_1 + X_3 + \epsilon_Y \\ X_2 &= \frac{1}{2}X_1 + \epsilon_2 \end{aligned}$$



Correlation screening would capture X_2 as a relevant feature.

Example

Consider a simple single-feature example:

$$X_1 \sim \text{Normal}(0, \sigma^2)$$

$$Y = X_1^2$$

Question: What is $\text{Corr}(X_1, Y)$?

Example continued

$$X_1 \sim \text{Normal}(0, \sigma^2)$$

$$\text{Corr}(X_1, Y) = \text{Corr}(X_1, X_1^2) = \frac{\text{Cov}(X_1, X_1^2)}{\sqrt{\text{Var}(X_1)\text{Var}(X_1^2)}}$$

$$\text{Cov}(X_1, X_1^2) = \mathbb{E}[(X_1 - \mathbb{E}[X_1])(X_1^2 - \mathbb{E}[X_1^2])] = \mathbb{E}[X_1(X_1^2 - \sigma^2)] = \mathbb{E}[X_1^3]$$

Since $\text{Normal}(0, \sigma^2)$ is symmetric distribution, $\mathbb{E}[X_1^3] = 0$

$$\text{Corr}(X_1, Y) = 0$$

Are X_1 and Y independent?

Recall: properties of correlation:

- $-1 \leq \rho_j, r_j \leq 1$
- Correlation invariant under linear transformation and unit-less
- If X_j and Y are independent, then $\rho_j = 0$

Are X_1 and Y independent?

- No: independence implies 0 correlation but converse not true
- Correlation captures linear relationship only
- Since $Y = X_1^2$ strong relationship but significant dependence
- Are there ways we can capture this non-linear relationship?

Mutual information

Most general way of determining independence/dependence:

$$MI(X_j, Y) = \mathbb{E}_{X_j, Y} \left[\log \frac{P(X, Y)}{P(X) \cdot P(Y)} \right]$$

Explicitly characterizes the relationship between the joint and marginal distributions

$$MI(X_j, Y) = 0 \Leftrightarrow X_j \text{ and } Y \text{ independent}$$

Mutual information for non-linear example

$$X_1 \sim \text{Normal}(0, \sigma^2)$$

$$Y = X_1^2$$

$$MI(X_1, Y) = \infty$$

As dependent as possible

But....

- Need to estimate/approximate entire joint and marginal distributions
- Estimating mutual information is extremely data intensive because you effectively need to estimate the entire joint and marginal distributions
- Also computationally intensive for continuous distributions or categorical distributions with large numbers of categories/values
- Speed-ups often needed to compute mutual information in practice (e.g. k-nearest neighbors, kernel density estimation, e.t.c.)

Mutual information for bike share data

```
from sklearn.feature_selection import mutual_info_regression
```

```
mi = mutual_info_regression(X, y, random_state=0)  
mi = pd.Series(mi, index=features).sort_values(ascending=False)
```

```
mi
```

```
atemp      0.464703  
temp       0.389542  
mnth       0.377056  
yr         0.274648  
season     0.216842  
weathersit  0.066363  
windspeed  0.055195  
hum        0.046375  
weekday    0.045011  
workingday 0.022315  
dtype: float64
```

Note: Absolute numbers difficult to interpret but rank has meaning

From mutual information to p-values: Permutation testing

- For general data types no standard asymptotic distribution for mutual information
- General approach: **Permutation testing**
 1. Permute responses/labels (or features)

$$(X_j^{(i)}, Y^{(\pi(i))})$$

where π is a permutation/ordering on $\{1, 2, \dots, n\}$

2. For $b = 1, 2, \dots, B$ permutations, estimate mutual information:

$$\widehat{MI}_j^b = \widehat{MI}(X_j, Y^{(\pi_b)})$$

3. Empirical null distribution:

$$\widehat{F}_0(t) = \frac{1}{B} \sum_{b=1}^B 1(\widehat{MI}_j^b \leq t)$$

4. Compute permuted p-value

$$p = \frac{1}{B} \sum_{b=1}^B 1(\widehat{MI}_j^b \geq \widehat{MI}_j^{obs})$$

Permutation testing in code

```
: from sklearn.feature_selection import mutual_info_regression

: def mi_permutation_test(X, y, n_perms=500, random_state=0):
    rng = np.random.default_rng(random_state)
    mi_obs = mutual_info_regression(X, y, random_state=random_state)
    mi_null = np.zeros((n_perms, X.shape[1]))

    for b in range(n_perms):
        y_perm = rng.permutation(y)
        mi_null[b] = mutual_info_regression(
            X, y_perm, random_state=random_state
        )

    p_vals = np.mean(mi_null >= mi_obs, axis=0)

    return mi_obs, p_vals

: mi_obs, mi_pvals = mi_permutation_test(X, y, n_perms=500)

mi_df = pd.DataFrame({
    "feature": features,
    "MI": mi_obs,
    "p_value": mi_pvals
}).sort_values("MI", ascending=False)
```

And applying multiple comparisons correction

```
: from statsmodels.stats.multitest import multipletests

    rej, p_adj, _, _ = multipletests(
        mi_df["p_value"],
        alpha=0.05,
        method="fdr_bh"
    )

    mi_df["p_adj"] = p_adj
    mi_df["significant"] = rej
    mi_df
```

Final selected feature list

	feature	MI	p_value	p_adj	significant
1	atemp	0.464703	0.000	0.000000	True
0	temp	0.389542	0.000	0.000000	True
6	mnth	0.377056	0.000	0.000000	True
5	yr	0.274648	0.000	0.000000	True
4	season	0.216842	0.000	0.000000	True
9	weathersit	0.066363	0.000	0.000000	True
3	windspeed	0.055195	0.034	0.042500	True
2	hum	0.046375	0.048	0.053333	False
7	weekday	0.045011	0.032	0.042500	True
8	workingday	0.022315	0.102	0.102000	False

- Weekday becomes significant most likely due to non-linear relationship
- Humidity no longer significant most likely due to loss of power with using mutual information
- p-values generally higher/less significant due to loss of power in estimating mutual information
- In complete contrast to correlation: mutual information extremely data and computational intensive (especially with permutation testing)

Is there a way to deal with the conditional vs marginal dependence issue?

Recall feature-wise subset selection $\tilde{S}$:

$$Y \perp X_j | X_{-j}, \text{ for all } j \in \tilde{S}^c$$

$$Y \not\perp X_j | X_{-j}, \text{ for all } j \in \tilde{S}$$

Solution: Replace marginal correlation with conditional correlation

Conditional/partial correlation

Rather than estimating

$$\rho_j := \text{Corr}(X_j, Y),$$

estimate *conditional/partial* correlation:

$$\rho_{j|-j} := \text{Corr}(X_j, Y|X_{-j}) = \frac{\text{Cov}(X_j, Y|X_{-j})}{\sqrt{\text{Var}(X_j|X_{-j})\text{Var}(Y|X_{-j})}},$$

Properties:

1. $-1 \leq \rho_{j|-j} \leq 1$
2. Also invariant under linear transformations
3. $Y \perp X_j | X_{-j} \Rightarrow \rho_{j|-j} = 0$

Understanding conditional correlation

$$X_j = \sum_{k \neq j} \beta_k X_k + \epsilon_j$$

$$Y = \sum_{k \neq j} \alpha_k X_k + \epsilon_Y$$

$$\rho_{j|-j} = \text{Corr}(\epsilon_j, \epsilon_Y)$$

- Regress both X_j and Y on X_{-j} and find correlation between residuals
- This is also how conditional/partial correlation is solved in practice
- In practice need to solve two linear regression problems
- We can apply a similar hypothesis testing framework with statistic

$$t_j = \widehat{\rho}_{j|-j} \sqrt{(n - p - 3) / (1 - \widehat{\rho}_{j|-j}^2)}$$

Conditional correlation in code: pre-processing

- Install necessary package and pre-process categorical features

```
: categorical_features = ['season', 'yr', 'mnth', 'holiday', 'weekday', 'workingday', 'weathersit']  
   numeric_features = ['temp', 'atemp', 'hum', 'windspeed']  
  
   # One-hot encode categorical features  
   df_cat = pd.get_dummies(df[categorical_features], drop_first=True)  
   df_num = df[numeric_features]  
  
   # Combine numeric and encoded categorical features  
   X = pd.concat([df_num, df_cat], axis=1)  
  
: pip install pingouin
```


Conditional correlation in code: model

```
from pingouin import partial_corr

y_col = 'cnt'

results = []

for feature in X.columns:
    # Conditioning set: all other features
    covars = [f for f in X.columns if f != feature]

    # Compute partial correlation
    res = partial_corr(data=pd.concat([X, df[y_col]], axis=1),
                      x=feature, y=y_col, covar=covars)
    results.append({
        'feature': feature,
        'partial_r': res['r'].values[0],
        'p_val': res['p-val'].values[0]
    })

# -----
# 3. Create results dataframe
# -----
results_df = pd.DataFrame(results)
# Sort by absolute partial correlation
results_df['abs_r'] = results_df['partial_r'].abs()
results_df = results_df.sort_values(by='abs_r', ascending=False).reset_index(drop=True)

# -----
# 4. Display results
# -----
print("Conditional correlation screening (conditioned on all other features):")
print(results_df[['feature', 'partial_r', 'p_val']])
```

Conditional correlation p-values

Conditional correlation screening (conditioned on all other features):

	feature	partial_r	p_val
0	yr	0.759493	2.112250e-136
1	season	0.327989	1.523906e-19
2	weathersit	-0.279209	2.231164e-14
3	windspeed	-0.204619	2.964111e-08
4	weekday	0.156085	2.557645e-05
5	hum	-0.120135	1.229600e-03
6	holiday	-0.095832	1.003315e-02
7	mnth	-0.084809	2.276281e-02
8	atemp	0.083551	2.486528e-02
9	workingday	0.062214	9.506706e-02
10	temp	0.053828	1.487710e-01

Conditional correlation p-values with corrections $\alpha = 0.05$

```
from statsmodels.stats.multitest import multipletests

# Extract p-values
pvals = results_df['p_val'].values

# Apply FDR correction
reject, pvals_corrected, _, _ = multipletests(pvals, alpha=0.03, method='fdr_bh')

results_df['p_val_fdr'] = pvals_corrected
results_df['significant_fdr'] = reject

# Filter features that are significant after FDR correction
significant_features_fdr = results_df[results_df['significant_fdr']].sort_values(by='abs_r', ascending=False)
print("Significant features after FDR correction:")
print(significant_features_fdr[['feature', 'partial_r', 'p_val', 'p_val_fdr']])
```

Significant features after FDR correction:

	feature	partial_r	p_val	p_val_fdr
0	yr	0.759493	2.112250e-136	2.323474e-135
1	season	0.327989	1.523906e-19	8.381485e-19
2	weathersit	-0.279209	2.231164e-14	8.180934e-14
3	windspeed	-0.204619	2.964111e-08	8.151304e-08
4	weekday	0.156085	2.557645e-05	5.626818e-05
5	hum	-0.120135	1.229600e-03	2.254266e-03
6	holiday	-0.095832	1.003315e-02	1.576638e-02
7	mnth	-0.084809	2.276281e-02	3.039089e-02
8	atemp	0.083551	2.486528e-02	3.039089e-02

Conditional correlation p-values with corrections $\alpha = 0.03$

```
from statsmodels.stats.multitest import multipletests

# Extract p-values
pvals = results_df['p_val'].values

# Apply FDR correction
reject, pvals_corrected, _, _ = multipletests(pvals, alpha=0.03, method='fdr_bh')

results_df['p_val_fdr'] = pvals_corrected
results_df['significant_fdr'] = reject

# Filter features that are significant after FDR correction
significant_features_fdr = results_df[results_df['significant_fdr']].sort_values(by='abs_r', ascending=False)
print("Significant features after FDR correction:")
print(significant_features_fdr[['feature', 'partial_r', 'p_val', 'p_val_fdr']])
```

Significant features after FDR correction:

	feature	partial_r	p_val	p_val_fdr
0	yr	0.759493	2.112250e-136	2.323474e-135
1	season	0.327989	1.523906e-19	8.381485e-19
2	weathersit	-0.279209	2.231164e-14	8.180934e-14
3	windspeed	-0.204619	2.964111e-08	8.151304e-08
4	weekday	0.156085	2.557645e-05	5.626818e-05
5	hum	-0.120135	1.229600e-03	2.254266e-03
6	holiday	-0.095832	1.003315e-02	1.576638e-02

General conclusions

- month and atemp borderline features, workingday and temp not selected
- weekday is significant
- Notably now we are not selecting as many highly correlated features due to the conditioning

Overall conclusions

- 3 different methods with different results
- We never know the *actual* ground truth
- Thoughts on which features you would recommend/select as significant?